

AI Agent:

Components and System

Wenbo Sun
School of Computing and Data Science, HKU

Autonomy and trust

Field	Task delegated to AI
1 Learning	Summarize a research article and explain its main ideas.
2 Communication	Read incoming emails and send replies in your name.
3 Travel	Plan a holiday and book flights and hotels with your credit card.
4 Software	Fix a bug in a live registration system, test it and deploy the change.
5 Finance	Monitor your investments and execute trades without consulting you.

What autonomy would you allow? Under what conditions would you trust it?

Agent definitions

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through executor.”

Russell & Norvig · Artificial Intelligence: A Modern Approach

Wooldridge & Jennings: autonomy, social ability, reactivity and proactiveness.

Software agents sense through data and interfaces, and act through executable operations.

Agent history

Period / anchor	Mechanism	Engineering requirement
1970s–1980s	Actors, messages, rules, plans	Explicit representations and coordination
1990s	Autonomous and rational agents	Goals, observations and action selection
1998 · Sutton & Barto	RL policies and reward-driven learning	Environment, trajectories and feedback
2015 · DQN	Deep action-value function	Learning from large interaction datasets

LLM-agent milestones

Anchor	Contribution to agent building
2020 · GPT-3	Natural-language task instructions and examples
2022 · MRKL / ReAct	External modules; alternating actions and observations
2023 · Toolformer	Learning API-use behavior from augmented text
Current applications	Persistent state, tool interfaces, Skills and runtime control

The LLM generates tokens; runtime code turns selected outputs into operations.

Agent landscape

System family	Typical observations	Typical actions
Coding agents	Files, test output, build logs	Edit files; run tests
Computer-use agents	Screenshots and UI state	Click, type, navigate
Research / support agents	Documents, API records, user replies	Search, inspect, ask, respond
Long-running assistants	Saved tasks, events, checkpoints	Resume work within a policy

Agent demonstration



Watch for

An observation

An executed action

A check on the result

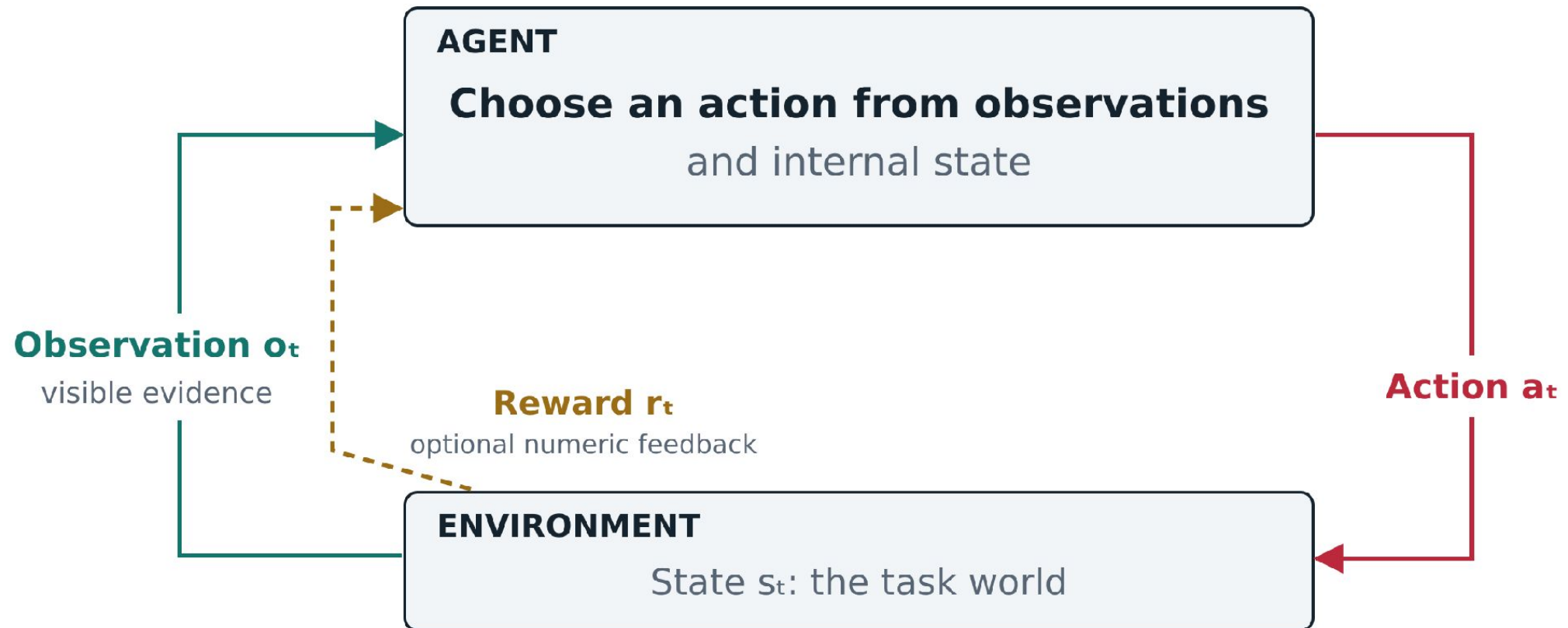
Coding application

OpenHands: get the Flask to-do application running.

Responsibility	Concrete requirement
Interpret	Understand the requested behavior of the to-do app
Observe	Inspect the application log and browser output
Act	Resolve the port conflict and restart the server
Verify	Reopen the page and check the requested behavior

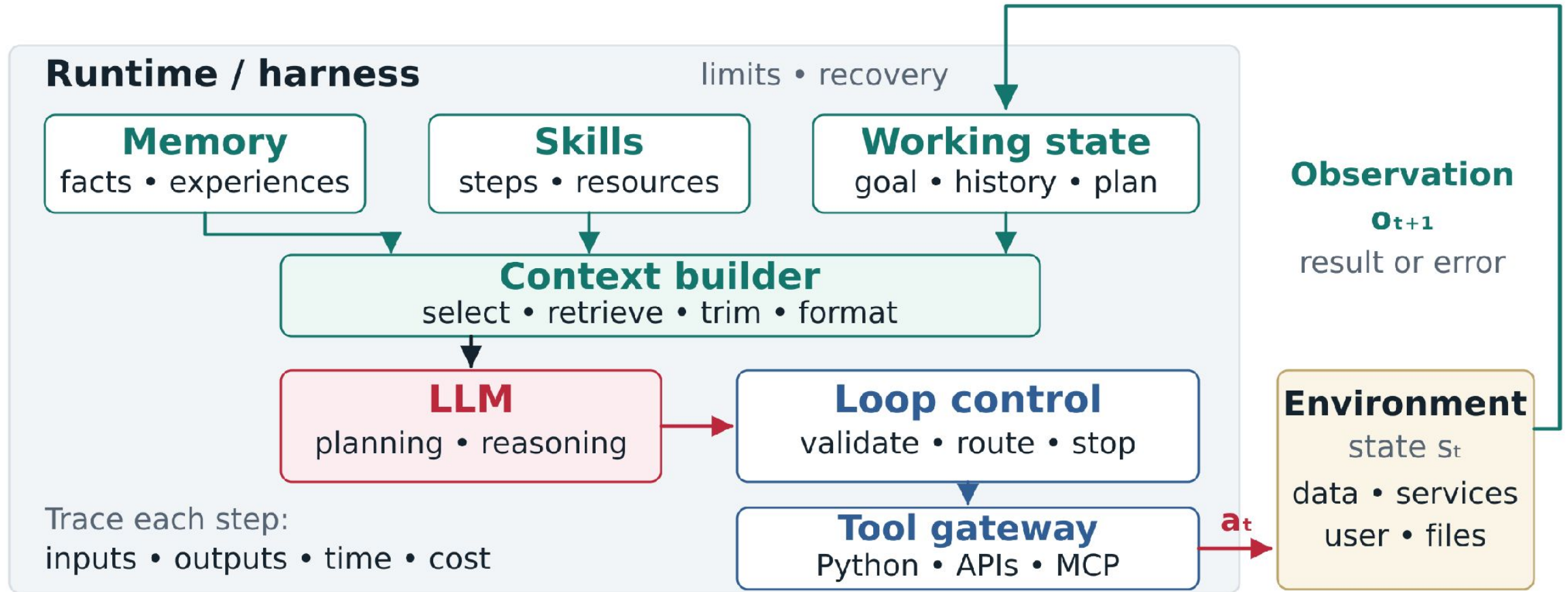
The model contributes a decision. The application must complete the interaction.

Agent–environment loop



Policy: $a_t \sim \pi(\cdot \mid o_t, z_t)$, where z_t is the agent's working state.

Agent architecture



Order support assistant

A user types a request. The assistant returns an answer supported by local records.

Function	Expected behavior
Order status	Look up a supplied order ID and report its stored status.
Delivery information	Show the stored policy for general delivery questions.
Support contact	Display the contact details for human support.
Incomplete or unsupported request	Ask for useful input or explain the available services.

Local model: llama3.2:1b. Records: SQLite. Interface: a notebook text box.

One agent iteration

Step	Artifact	Responsible component	Required capability
Assemble	Instructions, evidence and tool schemas	Context builder	Retrieval, selection and token budgeting
Decide	lookup_order order_id: ORD-1002	Model / planner	Instruction following, reasoning and structured generation
Execute	Checked service request	Runtime / harness and tool adapter	Validation, permissions and tool dispatch
Observe	Processing for 6 days from the order service	Service and state update	Result interpretation and evidence retention
Continue / stop	Next call, reply or escalation	Runtime / harness and model	Progress checks, execution budgets and stopping rules

Long context helps when relevant evidence reaches each model call.

Model decisions

The LLM interprets the supplied context and generates a proposed decision.

Part of the decision	Role in the order assistant
Input	Instructions, allowed output labels and the current user request.
Output	One intent label, encoded as a JSON object.
Application responsibility	Validate that output and decide whether execution may proceed.

Serve llama3.2:1b through Ollama. Measure its decisions on the task cases.

Planning and ReAct

Decision summary	Action	New observation
Identify the current status	lookup_order(ORD-1002)	Processing for 6 days
Find the applicable rule	lookup_policy("processing_delay")	Escalate after 5 days
Enough evidence to respond	Ask permission to escalate	User accepts or declines

New observations can change the next action; a plan still requires execution.

Context

Context is the information the model receives for one inference call.

Function

Select relevant input, assign message roles and fit the available token budget.

Order example

Classification instructions from a prompt file and the current user request.

Boundary

A database row or memory record affects the model only when included in its input.

Implementation: Python message construction, middleware or a LangGraph node.

Context contents

Context component	Concrete example
Instructions and examples	Supported intents, output rules, a few labeled requests
Current input and relevant history	User request plus unresolved task details
Tool definitions	Names, descriptions and argument schemas
Evidence and selected memory	Order result, source/time, confirmed preference
Activated Skills / plan	Relevant procedure and current task progress

https://docent.transluce.org/dashboard/b038912e-0133-4594-b093-92806f8ffb17/agent_run/0c69ca3e-3b3b-4e98-8026-916e90323208?transcript_id=5907a1f3-b191-44df-89e3-d4630e59afee&block_idx=3

Context budget

$$I + H + U + T + E + M + Y \leq W$$

I instructions H history

U current request T tool schemas

E evidence M memory / Skills

Y reserved generation

W available context window

Example: $W = 8,192$; $Y = 1,024$

Input budget $\leq 7,168$ tokens

Selection pressure

Retain the current goal and unresolved fields.

Prefer relevant, fresh evidence.

Measure token counts at the model interface.

Capacity, relevance and freshness are separate checks.

Context engineering

Operation	Implementation	Failure it addresses
Select	SQL lookup or a retrieval component	Irrelevant or absent evidence
Structure	Message roles and typed tool results	Mixed provenance or instructions
Reduce	Token-aware trim or summarization middleware	Growing input and distraction
Refresh	Re-query operational data when it expires	Stale status or policy

Run the selection policy in Python, LangChain middleware or a LangGraph node.

Context builder

```
def build_context(state, memory):  
    facts = select_fresh(state.observations)  
    prefs = memory.get(state.user_id)  
    turns = keep_complete_turns(  
        state.messages, token_budget=4000)  
    return format_messages(  
        instructions=POLICY,  
        history=turns,  
        evidence=facts,  
        preferences=prefs,  
        request=state.request)
```

Implementation points

Plain Python

A function called before inference.

LangChain

Model-call middleware.

LangGraph

A context-building node.

Pseudocode: helper policies belong to the application.

Memory

Memory retains information so the application can use it in a later step or task.

Responsibility	Concrete example
Retain task state	Keep the current intent, supplied ID and pending question.
Reuse selected information	Load a confirmed language preference or a relevant past case.
Control updates	Record the source, scope and freshness before reusing a value.

Today: one request dictionary. Later: a scoped store and persistent checkpoints.

Memory categories

Content	Support example	Read pattern
Episodic: experience	Earlier case, action, result and outcome	Retrieve a similar completed case
Semantic: knowledge	User-confirmed language preference	Exact lookup by user
Procedural: method	A delay-investigation procedure	Load the relevant procedure

External memory is editable data; model weights need not change.

Memory scope and storage

Scope	Typical data	Simple implementation
One request	Intent, selected ID and current evidence	Python dict or typed state
One conversation / task	Messages, pending question and progress	LangGraph checkpointer + SQLite/Postgres
Across conversations	Preferences, reusable cases and procedures	SQL tables, files or LangGraph Store

A vector index is an optional retrieval method, not a memory scope.

Memory records

Working state

```
{  
  "thread_id": "T7",  
  "intent": "order_status",  
  "order_id": null,  
  "awaiting": "order_id",  
  "step": 2  
}
```

Reusable memory

```
{  
  "user_id": "U42",  
  "language": "English",  
  "source": "user-confirmed",  
  "updated_at": "2026-09-24",  
  "expires_at": null  
}
```

Task state and confirmed preferences have different lifetimes.

Memory read–write lifecycle

Write

Observation → candidate → validate / deduplicate → scoped record.

Read

Filter by owner and freshness → retrieve → select for context.

Maintain

Correct or expire stale records; retain source and update time.

Schedule

Immediate write adds latency; background consolidation delays reuse.

Use SQL for keyed records and a vector index for similarity retrieval.

LangGraph memory

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
store.put(("users", "U42"), "preferences",
        {"language": "English"})
app = builder.compile(
    checkpointer=InMemorySaver(), store=store)
app.invoke(inputs,
    {"configurable": {"thread_id": "T7"}})
```

State and storage

Checkpoint
Snapshots for one thread.

Store
Records shared across threads.

These RAM backends lose data on restart.

Actual framework API fragment; builder and inputs are application objects.

Skills

Reusable task instructions, with references and optional executable helpers.

Package element	Example: investigate a failing test	Role in execution
Metadata	Name and short task description	Help select a relevant procedure
Instructions	Reproduce, inspect, change and verify	Guide how the task is performed
Optional resources	Test conventions and helper scripts	Load or execute when needed

Build with SKILL.md files and a loader, or Deep Agents SkillsMiddleware.

Skill package

File or directory	Purpose	Coding example
SKILL.md	Name, description and procedure	Investigate a failing test
scripts/	Optional executable helper	Run the smoke-test command
references/	Detailed material read on demand	Repository test conventions
assets/	Templates or other resources	Bug report template

Agent Skills defines files. A loader or SkillsMiddleware makes them available.

SKILL.md

```
---  
name: investigate-test-failure  
description: Diagnose a failing test and verify a repair.  
---  
# Procedure  
1. Reproduce the reported failure.  
2. Inspect the relevant code and test output.  
3. Make a focused change.  
4. Run the affected checks and record the result.  
  
See references/test-conventions.md for repository rules.
```

Metadata selects the Skill; instructions and references guide its use.

Example path: skills/investigate-test-failure/SKILL.md

Published Skill example

```
# Published file excerpt
name: pdf
...
# PDF Processing Guide
...
If you need to fill out a PDF form,
read FORMS.md and follow its
instructions.
```

Staged reading

1. Discover the PDF Skill.
2. Load its main instructions.
3. Follow the form-specific reference when the task requires it.

[Published source: anthropics/skills · skills/pdf/SKILL.md](#)

Skill loading

Stage	Context or action	Runtime responsibility
Discover	Names and descriptions	Find and index available Skills
Activate	Selected SKILL.md instructions	Load a versioned procedure
Use resources	Relevant reference or script result	Read or execute through an interface

SkillsMiddleware discovers metadata. read_file loads the selected procedure.

Environment and services

The environment holds external facts and produces the results of executed actions.

In today's system	Role
User input	Supplies the request and an explicit order ID when required.
SQLite records	Store fictional order statuses, delivery policy and contact details.
Service functions	Read a permitted record and return a reply with its evidence.

Python sqlite3 supplies the data interface. Students receive the service code.

Tools

A tool exposes an executable operation through a name, input contract and result.

Boundary	What happens
Describe	A schema states the operation and required arguments.
Execute	The host checks the request and invokes a function or service.
Observe	The returned value or error becomes evidence for the next step.

Build with Python functions, a framework tool wrapper or an MCP adapter.

Tool contract

Executable operation

```
def lookup_order(order_id: str):  
    """Read one demo order."""  
    return ORDERS.get(order_id,  
        {"error": "not_found"})  
  
# Observation  
{"order_id": "ORD-1001",  
 "status": "shipped"}
```

Model-visible schema

```
{"name": "lookup_order",  
 "description": "Read one order",  
 "parameters": {  
     "type": "object",  
     "properties": {  
         "order_id": {"type": "string"}},  
     "required": ["order_id"],  
     "additionalProperties": false  
 }}
```

Adapters: Python functions, LangChain tool wrappers or an MCP SDK.

Naive JSON tool protocol

```
# Action proposal
{"kind": "tool", "name": "lookup_order",
 "arguments": {"order_id": "ORD-1001"}}

# Terminal alternatives
{"kind": "final", "text": "Your order has shipped."}
{"kind": "ask_user", "text": "What is your order ID?"}
```

https://docent.transluce.org/dashboard/b038912e-0133-4594-b093-92806f8ffb17/agent_run/0c69ca3e-3b3b-4e98-8026-916e90323208?transcript_id=5907a1f3-b191-44df-89e3-d4630e59afee&block_idx=3

Parsing and dispatch

```
# Dispatcher branch: kind == "tool"
call = json.loads(raw_output)
validate(call, CALL) # jsonschema.validate
name = call["name"]
args = call["arguments"]

if name not in ALLOWED_TOOLS:
    raise ValueError("Unknown tool")
authorize(user, name, args)
result = ALLOWED_TOOLS[name](**args)
```

Separate checks

JSON syntax
Schema and types
Allowed operation
Permission and preconditions

Only then execute.

Use an allowlist and schema validator, or a framework tool wrapper.

Native function calling

```
from ollama import chat

reply = chat(model=MODEL, messages=messages,
             tools=[lookup_order])
calls = reply.message.tool_calls or []
if len(calls) != 1:
    raise ValueError("Expected one tool call")
call = calls[0]
name = call.function.name
args = call.function.arguments
```

API output

A structured call field supplies name and arguments.

The host validates and invokes the selected function.

Tool-capable model required.

Provider formats differ. This slide uses Ollama consistently.

Tool-result messages

```
messages.append(reply.message)
messages.append({
    "role": "tool",
    "tool_name": name,
    "content": json.dumps(result)
})
next_reply = chat(model=MODEL,
    messages=messages, tools=[lookup_order])
```

What changed?

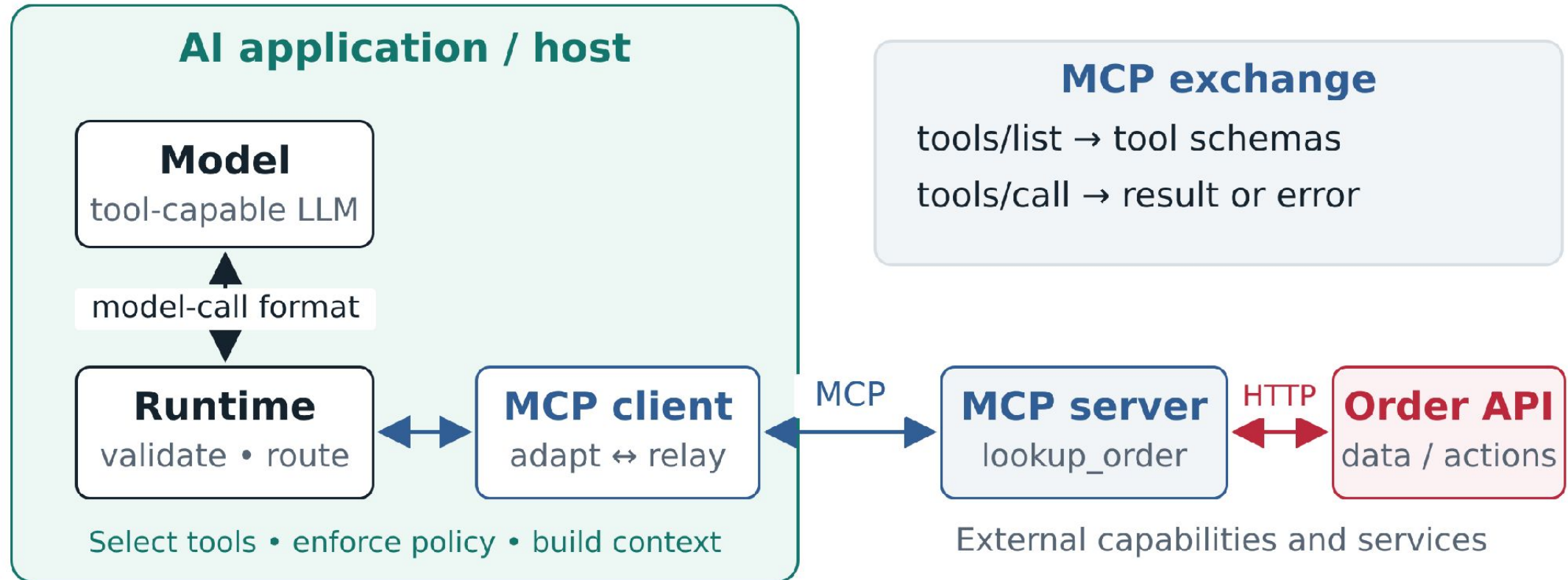
The host retained the action.

The service supplied an observation.

The next call sees both and can finish or act again.

Action → execution → observation → updated context → next model decision.

Model Context Protocol (MCP) architecture



The SDK handles the protocol. The harness controls tool execution.

MCP discovery and execution

Discovery

```
{"method": "tools/list"}
```

```
# Response includes  
name  
description  
inputSchema
```

```
# Host adapts selected schemas  
# to the model API.
```

Execution

```
{"method": "tools/call",  
 "params": {  
   "name": "lookup_order",  
   "arguments": {  
     "order_id": "ORD-1001"  
   }  
 }}
```

```
# Result becomes an observation.
```

MCP standardizes exchange; the host supplies control and permission policy.

Tool errors and preconditions

Condition	Concrete check or result
Wrong shape / name	Reject unknown fields, types or operations
Missing or invented ID	Require explicit input or verified task state
Unknown / inaccessible order	Return a structured not_found or forbidden result
Service unavailable	Record timeout/error; use a bounded policy

Schemas define form; the runtime and service enforce execution conditions.

Runtime / harness

The software that runs and constrains the model, action and observation loop.

Execution boundary	Software responsibility
Before inference	Assemble context and choose the model configuration
After model output	Validate the proposed action and dispatch an allowed operation
After execution	Record the observation and update the working state
Before continuing	Check completion, limits and whether human input is needed

Reading: LangChain, The Anatomy of an Agent Harness (2026)

Harness responsibilities

Responsibility	Concrete mechanism	Implementation example
Execution control	Branches, dispatch and stopping rules	Python loop or LangGraph graph
State and recovery	Persisted snapshots and resume input	LangGraph checkpointer + database
Limits and evidence	Attempts, deadlines and execution traces	Middleware, counters and LangSmith

Harness engineering measures changes to the software around inference.

Long-running work

Stage	Retained artifact	Purpose on the next run
Initialize	Feature list and environment startup script	Define what remains and how to start
Make progress	A focused change and executed checks	Establish a verifiable increment
Leave a handoff	Progress record and version-control history	Recover the current work state
Resume	Read the handoff and test the environment	Choose the next unfinished task

The runtime loads and checks retained artifacts when work resumes.

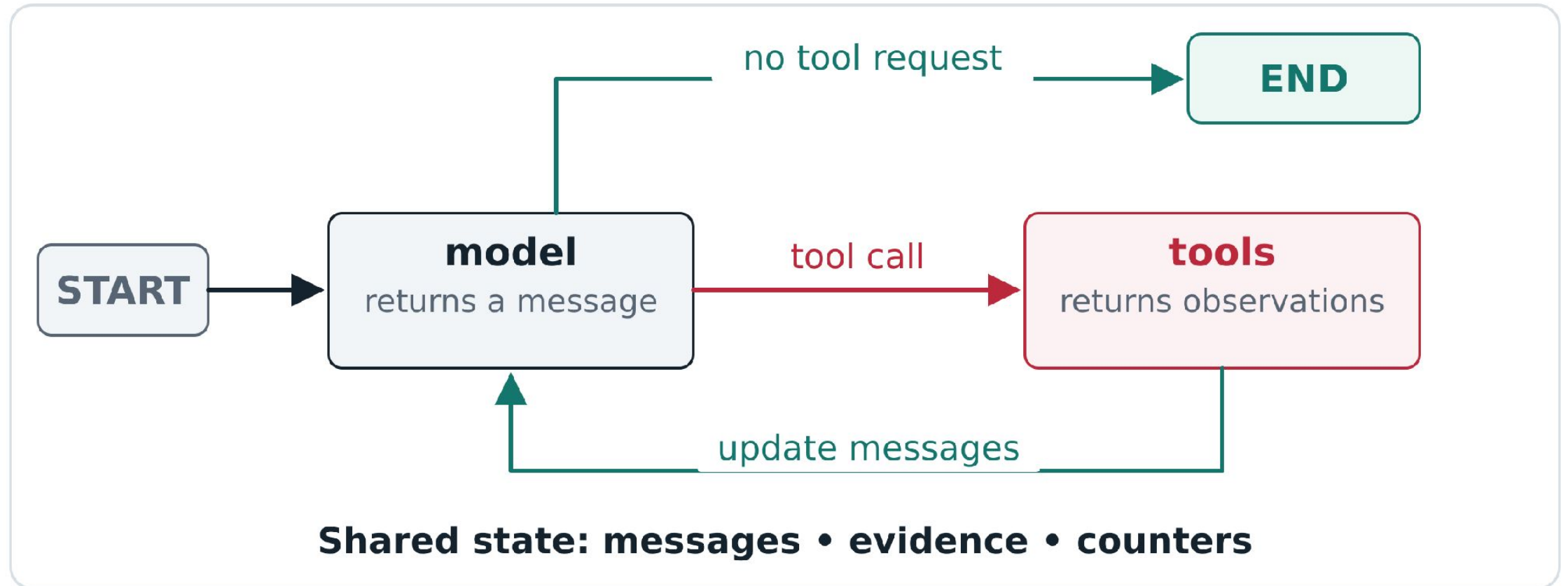
Case study: Anthropic, Effective harnesses for long-running agents (2025)

LangChain and LangGraph

Software	Place in the architecture	What it supplies
LangChain	Model/tool interfaces and agent harness APIs	Model adapters, tool wrappers and middleware
LangGraph	State and execution runtime	Nodes, edges, checkpoints and interrupts
Deep Agents	Packaged harness using LangChain and LangGraph	File tools, Skills and context management
LangSmith	Observability and evaluation	Traces, datasets and experiment comparisons

LangChain agents use LangGraph. Deep Agents adds packaged harness behavior.

LangGraph execution graph



The return edge makes a tool result available to a later model decision.

State and node functions

```
from langgraph.graph import MessagesState

model_with_tools = local_chat_model.bind_tools(
    [lookup_order])

def model_node(state: MessagesState):
    msgs = state["messages"]
    reply = model_with_tools.invoke(msgs)
    return {"messages": [reply]}

def route(state: MessagesState):
    last = state["messages"][-1]
    return "tool" if last.tool_calls else "done"
```

Node contract

Input

Current application state.

Output

A state update.

Reducer

Merges the message update into history.

An explicit state schema makes component inputs and outputs inspectable.

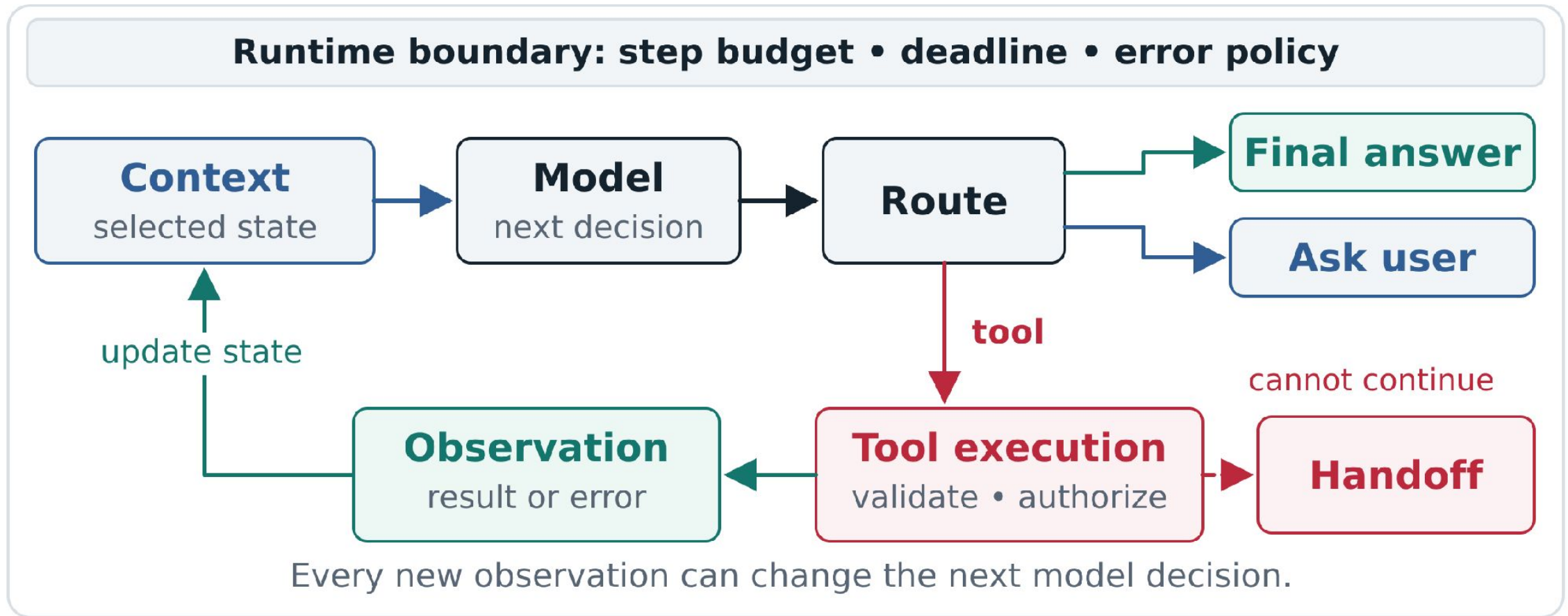
Graph construction

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode

g = StateGraph(MessagesState)
g.add_node("model", model_node)
g.add_node("tools", ToolNode([lookup_order]))
g.add_edge(START, "model")
g.add_conditional_edges("model", route,
                        {"tool": "tools", "done": END})
g.add_edge("tools", "model")
app = g.compile()
```

Nodes implement work. Edges implement execution order and feedback.

Bounded agent loop



Harness loop

```
for step in range(MAX_STEPS):  
    if deadline_expired():  
        return handoff("time budget")  
    d = decide(build_context(state), deadline=deadline)  
    if d.kind == "final":  
        return checked_answer(d, state)  
    if d.kind == "ask_user":  
        return ask_user(d.question)  
    observation = checked_dispatch(d, deadline=deadline)  
    state.record(d, observation)  
    if observation.unrecoverable:  
        return handoff(observation.reason)  
return handoff("step budget")
```

Pseudocode: the next iteration uses the observation stored by the previous one.

Checkpoints and resume

```
# Separate graph fragment; imports omitted.
def ask_id(state):
    value = interrupt("Please provide an order ID")
    return {"order_id": value}

clarify_app = clarify_graph.compile(
    checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "T7"}}
clarify_app.invoke({"order_id": None}, cfg)
clarify_app.invoke(
    Command(resume="ORD-1001"), cfg)
```

Recovery contract

Prerequisites

State includes order_id.

Graph: START → ask_id → END.

First invoke pauses.

Resume supplies the value.

Validate it before use.

InMemorySaver is volatile.

A durable LangGraph checkpointer preserves execution state across restarts.

Support chatbot

Type one request in a text field and press Send.

Request	Expected behavior
Parcel status	Read whether a supplied order is shipped, processing or delivered.
Delivery policy	Show the stored policy: 3-5 working days.
Further support	Show the stored contact: support@example.com.

Order facts come from SQLite. General answers come from the local model.

Demand analysis

Question	Decision to record
User and outcome	Who asks, and what reply would help them?
Evidence of success	Which record or answer would show success?
Information	What must the user supply? What belongs in the data?
Responsibility	Which decision needs the model? Which belongs in Python?
Failure and help	What happens without an ID or with a failed model call?
Resources	How many model calls and how much waiting are acceptable?

Use concrete requests to decide what the application must do.

Group specification

An order request

Write the input, the required ID and the record that supports the reply.

Missing information

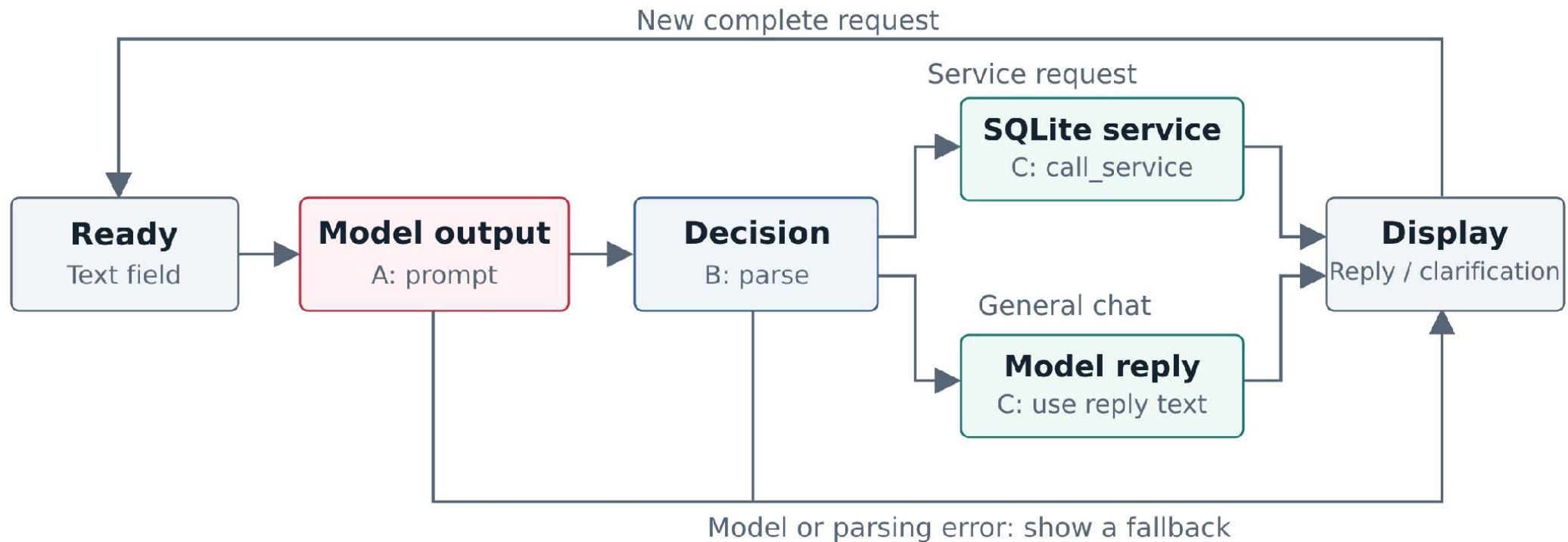
Write the clarification. Does the next submission need the full request?

A general question

Write an example the model can answer without reading order data.

Single-turn conversation

Each submission starts with the current request and the prompt.



After an ID question, resend "Track ORD-1001". No previous ID is remembered.

Notebook structure

Part	Provided	Student work
1 Data loading	One block loads JSON files into SQLite	Run it and inspect the records
2 Helpers	Two functions for the model and services	Run the block unchanged
3 Tasks A, B and C	A contract and a test block for each task	Write the prompt and two functions

The complete request path is visible in the final button callback.

Data loading

JSON files are the editable source. Part 1 fills the demo SQLite database.

Data	SQLite table	Example
Parcel records	orders(order_id, status)	ORD-1001: shipped
Policy and contact	info(name, text)	delivery_info: 3-5 working days

Add data_extra.json beside the notebook:

```
{"orders": [{"order_id": "ORD-1004", "status": "processing"}]}
```

Rerun Part 1, then ask about ORD-1004. No helper changes are needed.

Supplied functions

Function	Input	Output
<code>ask_model(user_text)</code>	Current request	Raw JSON text from llama3.2:1b
<code>call_service(intent, user_text)</code>	Service label and original request	A database reply or clarification
<code>json.loads(raw)</code>	JSON text	A Python value, or ValueError

The order service already checks the ID and reads its status.

Students use these interfaces without writing HTTP or SQL code.

Model output

Path	Accepted output
Parcel status	<code>{"intent":"order_status"}</code>
Delivery policy	<code>{"intent":"delivery_info"}</code>
Support contact	<code>{"intent":"contact_support"}</code>
General chat	<code>{"intent":"chat","reply":"Hello! How can I help?"}</code>

Service facts are supplied after the model decision.

Task A requests this format. Task B checks what actually arrived.

Task A: prompt

Write the instructions in `prompt_student.txt`.

Intent selection

Define the four labels and give examples, including a missing order ID.

Output format

Require the agreed JSON shape for a service or a general-chat reply.

Reply boundaries

Leave order facts to the services. Keep general answers short.

Rerun the prompt-loading cell after saving your file.

Task A: prompt check

Try this request	Expected intent
Track ORD-1001	order_status
Where is my parcel?	order_status
How long does delivery take?	delivery_info
I need a person	contact_support
What is a Python list?	chat, with a short reply

The optional live check tests one request. Try different wording in the UI.

Task B: parser

Implement `parse_output(raw)`: JSON text in, checked dictionary out.

Check	Required behavior
JSON	Parse the text using <code>json.loads</code> .
Intent	Require a dictionary and one supported intent label.
Fields	Service: intent only. Chat: intent and a non-empty text reply.
Invalid output	Raise <code>ValueError</code> so the UI can show a fallback.

A valid shape can still contain the wrong intent. Test meaning separately.

Task B: parser check

Recorded model output	Expected result
<code>{"intent":"order_status"}</code>	Accept
<code>{"intent":"chat","reply":"Hello!"}</code>	Accept
not JSON	Reject
<code>{"intent":"refund"}</code>	Reject unknown intent
<code>{"intent":"chat","reply":""}</code>	Reject empty chat reply

Eight parser checks run without a model. All must pass before integration.

Task C1: service selection

Implement `make_reply(decision, user_text)` and return one reply string.

Decision	Required behavior
Service intent	Call <code>call_service</code> with the intent and original request.
General chat	Return the model's reply without a service call.
Missing or unknown order ID	Use the clarification returned by the supplied service.

The eight reply checks use given decisions, so no model is needed.

Task C2: Integration

The supplied Send button connects the functions in order.

Step	Visible call	What happens
1	<code>ask_model(user_text)</code>	Use Task A's prompt and get raw text
2	<code>parse_output(raw)</code>	Task B checks the decision
3	<code>make_reply(decision, user_text)</code>	Task C chooses a service or chat reply
4	Display the returned text	Wait for a new complete request

One submission uses at most one model call and one service call.

Completion checks

Check	What to demonstrate
Task checks	Prompt written, eight parser checks and eight reply checks passed.
Live UI	Try parcel status, delivery policy, support contact and general chat.
Missing ID	Get a clarification, then resend a complete request.
Extra data	Add a parcel in data_extra.json, reload Part 1 and query it.

Record one model failure or limitation to discuss after implementation.

Failure localization

The program runs. The reply is wrong. Where would you investigate first?

Observed artifact	Value
User request	Track ORD-1001
Model output	<code>{"intent": "delivery_info"}</code>
Validation	The JSON object satisfies the schema
Application reply	Demo delivery normally takes 3–5 business days.

Discuss using context, model decision, request state and service evidence.

Conversation state

User: Where is my parcel?

Assistant: Please resend your request with an order ID.

User: ORD-1001

What should happen next?

Design question

What must survive between submissions?

Where should it be stored?

What should enter the next context?

What must change to resume a task across multiple submissions?

Hands-on synthesis

System role	What the first application contains
Goal and evidence	A user outcome and records that can support the reply
Model	A learned interpretation of the current request
Context and state	Selected instructions, current input and checked request fields
Runtime and environment	Validated execution over read-only services and SQLite data

Model capability becomes an application through information and control flow.

Testing strategy

What might the shared cases miss?

Test layer	Support example	Evidence
Input coverage	Paraphrases, malformed IDs and unsupported tasks	Held-out cases
Boundary behavior	Timeout, malformed output and missing record	Fault injection
End-to-end quality	Full request through model and services	Correct, supported outcome
Regression	Same cases after a prompt or model change	Versioned comparisons

Valid JSON with the wrong intent is an interpretation failure.

Fault tolerance

What should happen when a component fails?

Failure	Policy	Why
Missing required ID	Clarify	The user must supply the fact
Malformed model output	Reject; optional bounded repair later	Protect the program contract
Read-only service timeout	Limited retry, then fallback	Transient failure may recover
Unknown / forbidden order	Contact or rejection	Do not invent evidence

Termination and retry safety

```
# Policy pseudocode; app-defined helpers
MAX_MODEL_CALLS = 4
MAX_TOOL_ATTEMPTS = 2
DEADLINE_SECONDS = 10

stop_if_budget_exhausted(state)
retry_only_transient_read_failures()

# A future mutation needs a stable operation ID.
# A timeout does not prove that it failed.
```

Review question

Can this request keep running or repeat an effect?

A checkpoint restores state.

The external service must establish whether an action already committed.

Policy pseudocode: limits are application configuration choices.

Latency budget

$$\begin{aligned} L_{\text{request}} = & L_{\text{queue}} + L_{\text{context}} \\ & + \sum L_{\text{model_calls}} \\ & + \sum L_{\text{tool_calls}} + L_{\text{render}} \end{aligned}$$

For one model call, approximately:
 $L_{\text{model}} \approx L_{\text{prefill}} + N_{\text{out}} / r_{\text{decode}}$

Record cold and warm runs separately.
Compare p50 and p95 on the same workload.

Review question

Where does the user wait?

Long context affects prefill.
Long output affects decode.
Extra loop turns add calls.
A slow tool can dominate.

Streaming improves responsiveness; the execution work remains.

Cost accounting

Hosted inference; rates per million tokens

$$C_{\text{infer}} = \sum (T_{\text{in}} \times p_{\text{in}} + T_{\text{out}} \times p_{\text{out}}) / 1e6$$

Local inference: allocate resources over workload

$$C_{\text{infer}} = \text{allocated_machine_and_ops_cost} / N$$

$$C_{\text{request}} = C_{\text{infer}} + C_{\text{external_services}}$$

Retries and extra reasoning are additional work.

Review question

Which resources increase when the application improves?

Count all calls and tokens.

Include setup, idle capacity and operating costs where relevant.

Local inference still consumes machine capacity and operating resources.

Economic model

Expected total = inference + human handling + expected error loss

1,000 requests · illustrative values	Candidate A	Candidate B
Inference cost	HK\$10	HK\$40
Wrong answers / handoffs	50 / 50	20 / 30
Assumed p95 latency	1.2 s	3.8 s
Total expected cost	HK\$1,260	HK\$590

Assume HK\$5 per handoff and HK\$20 expected loss per wrong answer.

Question: which candidate is better if the latency limit is 2 seconds? All figures are hypothetical.

Model selection

Which model configuration satisfies the contract?

Gate	Experiment or measurement
Feasible deployment	CPU memory, model interface, schema/tool support
Required quality	Held-out requests, clarification and unsupported cases
Latency and reliability	Cold/warm p50/p95, timeout and error rate
Economics	Compute, external services, handoffs and expected error loss

Test-time compute

What extra work could fix the observed failure?

Mechanism	Useful signal	Cost / limitation
More reasoning budget	Difficult decision under adequate evidence	More generation; benefit is task-dependent
Sample + verify	A reliable way to select a better candidate	Extra calls plus verifier work
Bounded revision	Specific validation or task feedback	Another attempt; errors may repeat

Extra computation cannot replace a missing order ID or an unavailable source.

Component boundaries

Which responsibilities should change together?

Responsibility	Current starting point	Possible later boundary
Context selection	build_messages	Context builder with retrieval and token budgeting
Memory and state	A request dictionary	Scoped store plus a thread checkpointer
Reusable procedures	Instructions in the prompt	Skill discovery and loading
Service execution	call_service and run_request	Tool adapter and explicit execution policy

Extension design

How would we add a new capability without rewriting everything?

New requirement	Information or execution behavior to add
Accept an ID-only follow-up	Thread state, input merge and resume route
Use current delivery policy	Retriever / service + context builder
Reuse a specialized procedure	Skill registry + loading policy
Add a refund operation	Tool contract + authorization + effect/retry policy

Part II roadmap

Capability	What will be added to the first system
Evidence access	Retrieval, source selection and grounded answers
Adaptive action	Tool selection, observations and bounded agent loops
Continuity and reuse	Persistent state, memory and Skills
System evaluation	Failure suites, traces and cost/quality comparisons

Keep the architecture and contracts stable as capabilities become richer.